

Casos de Teste

Sistema Analítico Baseado em Evidências (Evidence-First Analytical System)

1. Objetivo: Este documento define a estratégia e os casos de teste para o Sistema Analítico Baseado em Evidências.

A suíte de testes verifica se o sistema é capaz de:

- Direcionar perguntas analíticas gerais para as capacidades suportadas;
- Executar operações analíticas determinísticas;
- Preservar distinções importantes nos dados de origem;
- Rejeitar perguntas não suportadas;
- Evitar suposições semânticas não fundamentadas;
- Auditar alegações analíticas de forma independente;
- Detectar resultados analíticos adulterados;
- Preservar a reprodutibilidade e a rastreabilidade.

A estratégia de testes está alinhada aos principais princípios de confiabilidade do sistema: computação determinística, geração de evidências, validação independente, limitações explícitas e rastreabilidade.

2. Organização dos Testes: Os testes estão divididos em três categorias

- Testes unitários: Verificam componentes individuais em isolamento, incluindo roteamento de habilidades, cálculos de evidências e relatórios de qualidade.
- Testes de integração: Verificam se múltiplos componentes funcionam em conjunto ao longo do fluxo analítico.
- Testes adversariais: Tentam expor comportamentos que poderiam levar a conclusões sem sustentação, transformações silenciosas de dados ou perda de rastreabilidade.

3. Ambiente de Teste

A suíte de testes pressupõe que o projeto foi instalado em modo editável:

```
pip install -e "[dev]"
```

Os testes são executados com:

```
pytest -v
```

Os testes analíticos utilizam o fluxo de dados Bronze/Silver/Gold do projeto.

4. Catálogo de Casos de Teste

TC-UNIT-001 - Roteamento de pico de riqueza

- **Categoria:** Unitário
- **Componente:** Registro de Habilidades (*Skill Registry*)
- **Objetivo:** Verificar se uma pergunta sobre o mês com a maior riqueza de espécies observada é direcionada para a habilidade analítica correspondente.
- **Entrada:** *"Qual mês apresentou a maior riqueza observada de espécies de aves?"*
- **Resultado esperado:** O registro identifica a habilidade peak_richness.
- **Condição de falha:** A pergunta não é roteada ou é direcionada para uma habilidade não relacionada.

TC-UNIT-002 - Roteamento de abundância

- **Categoria:** Unitário
- **Componente:** Registro de Habilidades
- **Objetivo:** Verificar se uma pergunta sobre abundância quantitativa é direcionada para a análise de abundância.
- **Entrada:** *"O que o conjunto de dados mostra sobre a abundância quantificada?"*
- **Resultado esperado:** O registro identifica abundance_summary.

TC-UNIT-003 - Roteamento de esforço

- **Categoria:** Unitário
- **Componente:** Registro de Habilidades
- **Objetivo:** Verificar se perguntas sobre esforço de amostragem são direcionadas para a análise de esforço.
- **Entrada:** *"Existem diferenças no esforço de amostragem?"*
- **Resultado esperado:** O registro identifica effort_discrepancy.

TC-UNIT-004 - Roteamento não suportado

- **Categoria:** Unitário
- **Componente:** Registro de Habilidades
- **Objetivo:** Verificar se perguntas fora do domínio analítico suportado não são atribuídas a nenhuma habilidade.
- **Entrada:** *"Como estarão as populações de aves em 2050?"*
- **Resultado esperado:** Nenhuma habilidade é selecionada.

5. Testes do Motor de Evidências (Evidence Engine)

TC-ENGINE-001 - Alegação de pico de riqueza

- **Categoria:** Unitário

- **Componente:** Motor de Evidências
- **Objetivo:** Verificar se o cálculo de pico de riqueza produz uma alegação de evidência estruturada.
- **Resultado esperado:** A alegação contém ID da alegação, nome da análise, fonte dos dados, resultado e metadados de auditoria. O resultado contém o mês de pico, lista de meses de pico, riqueza observada e indicador de empate.

TC-ENGINE-002 - Preservação de empates

- **Categoria:** Unitário
- **Componente:** Motor de Evidências
- **Objetivo:** Verificar se o sistema não seleciona arbitrariamente apenas um mês quando múltiplos meses possuem a mesma riqueza máxima.
- **Resultado esperado:** Em caso de empate: is_tied = true e mais de um mês presente em peak_months. Se não houver empate: is_tied = false e exatamente um mês presente.

TC-ENGINE-003 - Preservação de abundância qualitativa

- **Categoria:** Unitário
- **Componente:** Motor de Evidências
- **Objetivo:** Verificar se os registros de presença qualitativa permanecem distinguíveis da abundância quantitativa.
- **Resultado esperado:** A alegação gerada reporta separadamente: abundância quantificada, registros qualitativos e total de registros.

TC-ENGINE-004 - Preservação de contagem de indivíduos NULA

- **Categoria:** Unitário / Integridade de Dados
- **Componente:** Dados Silver
- **Objetivo:** Verificar se valores ausentes de individualCount não são convertidos silenciosamente em zero.
- **Resultado esperado:** Linhas com contagens quantitativas ausentes permanecem ausentes.
- **Justificativa:** Uma contagem ausente não fornece evidência de que zero indivíduos foram observados.

TC-ENGINE-005 - Proteção na representação de esforço

- **Categoria:** Unitário / Robustez
- **Componente:** Motor de Evidências
- **Objetivo:** Verificar se medições de esforço oriundas de estruturas de dados diferentes não são tratadas automaticamente como equivalentes.
- **Campos relevantes:** sampleSizeValue, sampleSizeUnit, samplingEffortValue, samplingEffortUnit

- **Resultado esperado:** A alegação registra equivalence_assumed = false e comparison_performed = false. O sistema preserva as representações separadas e registra a limitação de interpretação.

TC-ENGINE-006 - Reprodutibilidade da alegação

- **Categoria:** Unitário
- **Componente:** Motor de Evidências
- **Objetivo:** Verificar se a execução repetida da mesma operação analítica produz o mesmo identificador de alegação.
- **Resultado esperado:** Duas execuções com análise e parâmetros idênticos geram o mesmo claim_id.

TC-ENGINE-007 - Metadados de auditoria

- **Categoria:** Unitário
- **Componente:** Motor de Evidências
- **Objetivo:** Verificar se o Motor de Evidências não declara incorretamente seu próprio resultado como auditado independentemente.
- **Resultado esperado:** A alegação inicial contém execution_status = SUCCESS, is_reproducible = true e audit_status = PENDING.

6. Testes de Qualidade

TC-QUALITY-001 - Disponibilidade do relatório de qualidade

- **Categoria:** Unitário
- **Componente:** Camada de Qualidade
- **Objetivo:** Verificar se o relatório de qualidade determinístico é gerado.
- **Resultado esperado:** O arquivo relatorio_qualidade.json existe.

TC-QUALITY-002 - Estrutura do relatório de qualidade

- **Categoria:** Unitário
- **Componente:** Camada de Qualidade
- **Objetivo:** Verificar se o relatório de qualidade contém os campos estruturais obrigatórios.
- **Campos esperados:** total_events, total_occurrences, total_inventories, issues, is_valid_for_analysis, analysis_limitations.

TC-QUALITY-003 - Alerta de esforço

- **Categoria:** Unitário
- **Componente:** Camada de Qualidade
- **Objetivo:** Verificar se as diferentes representações de esforço do conjunto de dados são identificadas explicitamente.
- **Resultado esperado:** O relatório contém EFFORT_FIELDS_PRESENT.

TC-QUALITY-004 - Detecção de múltiplos protocolos

- **Categoria:** Unitário
- **Componente:** Camada de Qualidade
- **Objetivo:** Verificar se múltiplos protocolos de amostragem são detectados.
- **Resultado esperado:** O relatório contém MULTIPLE SAMPLING PROTOCOLS.

7. Testes de Integração

TC-INT-001 - Fluxo completo de riqueza

- **Categoria:** Integração
- **Objetivo:** Verificar a execução ponta a ponta (*end-to-end*) de uma pergunta de riqueza suportada.
- **Fluxo:** Pergunta do usuário → Orquestrador → Roteamento de habilidade → Análise determinística → Alegação de evidência → Auditoria → Relatório.
- **Resultado esperado:** A resposta possui status SUCCESS, ID de execução, habilidade selecionada, alegação de evidência, status de auditoria e arquivo de relatório.

TC-INT-002 - Fluxo completo de abundância

- **Categoria:** Integração
- **Objetivo:** Verificar a execução completa de uma pergunta de abundância.
- **Resultado esperado:** O fluxo é concluído com sucesso, produzindo uma alegação de evidência e o resultado da auditoria.

TC-INT-003 - Fluxo completo de esforço

- **Categoria:** Integração
- **Objetivo:** Verificar se uma pergunta de esforço atinge a análise correspondente e preserva a proteção semântica.
- **Resultado esperado:** equivalence_assumed = false.

TC-INT-004 - Fluxo não suportado

- **Categoria:** Integração
- **Objetivo:** Verificar se uma pergunta não suportada é rejeitada antes que ocorra qualquer processamento analítico indevido.
- **Resultado esperado:** status = UNSUPPORTED.

8. Testes de Auditoria Independente

TC-AUDIT-001 - Verificação de pico de riqueza

- **Categoria:** Integração
- **Objetivo:** Verificar se uma alegação de pico de riqueza recalculada independentemente é confirmada.
- **Resultado esperado:** audit_status = VERIFIED.

TC-AUDIT-002 - Verificação de abundância

- **Categoria:** Integração
- **Objetivo:** Verificar a validação independente dos cálculos de abundância.
- **Resultado esperado:** audit_status = VERIFIED.

TC-AUDIT-003 - Verificação de esforço

- **Categoria:** Integração
- **Objetivo:** Verificar se a alegação de evidência de esforço pode ser checada de forma independente.
- **Resultado esperado:** audit_status = VERIFIED.

TC-AUDIT-004 - Detecção de resultado adulterado

- **Categoria:** Integração / Robustez
- **Objetivo:** Verificar se o auditor não confia cegamente em uma alegação analítica.
- **Procedimento:**
 1. Gerar uma alegação de pico de riqueza válida.
 2. Modificar o resultado calculado.
 3. Submeter a alegação modificada ao auditor.
 4. Comparar a alegação com o recálculo independente.
- **Resultado esperado:** audit_status = FAILED.

9. Testes Adversariais

TC-ADV-001 - Pergunta induzida (*Leading question*)

- **Objetivo:** Verificar se a premissa trazida pelo usuário não se sobrepõe aos dados observados.
- **Entrada:** *"Por que dezembro de 2019 teve o maior pico de riqueza?"*
- **Comportamento esperado:** O sistema calcula o pico observado de forma independente em vez de aceitar a premissa como fato.

TC-ADV-002 - Presença qualitativa versus abundância

- **Objetivo:** Verificar se registros de presença qualitativa não são interpretados como abundância quantitativa.
- **Comportamento esperado:** O sistema mantém as informações qualitativas e quantitativas separadas.

TC-ADV-003 - Pergunta não suportada

- **Objetivo:** Verificar se o sistema recusa perguntas fora do seu escopo suportado.
- **Entrada:** *"Qual é a temperatura média da água na Amazônia?"*
- **Comportamento esperado:** Retornar UNSUPPORTED.

TC-ADV-004 - Proteção de esforço de amostragem

- **Objetivo:** Verificar se diferentes representações de esforço não são igualadas automaticamente.
- **Comportamento esperado:** equivalence_assumed = false e comparison_performed = false.

TC-ADV-005 - Auditoria independente

- **Objetivo:** Verificar se as alegações analíticas podem ser recalculadas independentemente.
- **Comportamento esperado:** VERIFIED para uma alegação válida.

TC-ADV-006 - Alegação adulterada

- **Objetivo:** Verificar se um resultado analítico alterado é detectado.
- **Comportamento esperado:** Retornar FAILED.

TC-ADV-007 - Proteção contra contagem ausente

- **Objetivo:** Verificar se valores ausentes de individualCount não se transformam em zero.
- **Comportamento esperado:** Valores ausentes permanecem ausentes.

TC-ADV-008 - Limitação explícita de esforço

- **Objetivo:** Verificar se a análise de esforço registra a limitação associada à comparabilidade semântica.
- **Comportamento esperado:** A alegação contém uma interpretação e limitação explícitas.

TC-ADV-009 - Estado pré-auditoria

- **Objetivo:** Verificar se uma alegação recém-gerada não é marcada automaticamente como auditada independentemente.
- **Comportamento esperado:** audit_status = PENDING antes da execução do auditor.

TC-ADV-010 - ID de alegação reprodutível

- **Objetivo:** Verificar se a mesma operação analítica gera um identificador de alegação reprodutível.
- **Comportamento esperado:** Execuções repetidas produzem o mesmo claim_id.

10. Critérios de Aceitação

O sistema é considerado pronto para demonstração quando:

- Todos os testes unitários passarem;
- Todos os testes de integração passarem;
- Todos os testes adversariais passarem;
- Perguntas não suportadas forem rejeitadas;

- Os cálculos críticos forem determinísticos;
- As alegações críticas puderem ser auditadas independentemente;
- Alegações adulteradas forem detectadas;
- Valores quantitativos ausentes não forem convertidos silenciosamente em zero;
- Diferentes representações de esforço de amostragem não forem tratadas como equivalentes sem evidências;
- As alegações possuírem identificadores reproduzíveis;
- As limitações analíticas forem registradas explicitamente;
- O fluxo completo pode ser executado a partir do repositório sem intervenção manual.

```

tests/adversarial/test_adversarial.py::test_leading_question_does_not_override_observed_result PASSED [ 3%]
tests/adversarial/test_adversarial.py::test_qualitative_presence_is_not_treated_as_quantitative_abundance PASSED [ 6%]
tests/adversarial/test_adversarial.py::test_unsupported_question_is_rejected PASSED [ 9%]
tests/adversarial/test_adversarial.py::test_effort_representations_are_not_equated PASSED [ 12%]
tests/adversarial/test_adversarial.py::test_independent_audit_verifies_engine_claim PASSED [ 15%]
tests/adversarial/test_adversarial.py::test_auditor_rejects_tampered_claim PASSED [ 18%]
tests/adversarial/test_adversarial.py::test_missing_count_is_not_zero PASSED [ 21%]
tests/adversarial/test_adversarial.py::test_effort_claim_explicitly_records_limitation PASSED [ 24%]
tests/adversarial/test_adversarial.py::test_claim_requires_audit_before_being_considered_verified PASSED [ 27%]
tests/adversarial/test_adversarial.py::test_claim_id_is_reproducible PASSED [ 30%]
tests/integration/test_audit.py::test_peak_richness_is_independently_verified PASSED [ 33%]
tests/integration/test_audit.py::test_abundance_is_independently_verified PASSED [ 36%]
tests/integration/test_audit.py::test_effort_claim_is_independently_verified PASSED [ 39%]
tests/integration/test_audit.py::test_auditor_detects_tampered_peak_result PASSED [ 42%]
tests/integration/test_pipeline.py::test_peak_richness_pipeline_completes PASSED [ 45%]
tests/integration/test_pipeline.py::test_abundance_pipeline_completes PASSED [ 48%]
tests/integration/test_pipeline.py::test_effort_pipeline_completes PASSED [ 51%]
tests/integration/test_pipeline.py::test_unsupported_pipeline_is_rejected PASSED [ 54%]
tests/unit/test_evidence_engine.py::test_peak_richness_returns_valid_claim PASSED [ 57%]
tests/unit/test_evidence_engine.py::test_peak_richness_handles_ties PASSED [ 60%]
tests/unit/test_evidence_engine.py::test_abundance_preserves_qualitative_records PASSED [ 63%]
tests/unit/test_evidence_engine.py::test_missing_individual_count_is_not_converted_to_zero PASSED [ 66%]
tests/unit/test_evidence_engine.py::test_effort_claim_preserves_distinct_representations PASSED [ 69%]
tests/unit/test_evidence_engine.py::test_claim_has_reproducibility_metadata PASSED [ 72%]
tests/unit/test_evidence_engine.py::test_claim_id_is_stable_for_same_analysis PASSED [ 75%]
tests/unit/test_quality.py::test_quality_report_exists PASSED [ 78%]
tests/unit/test_quality.py::test_quality_report_has_expected_structure PASSED [ 81%]
tests/unit/test_quality.py::test_quality_report_contains_effort_warning PASSED [ 84%]
tests/unit/test_quality.py::test_quality_report_detects_multiple_protocols PASSED [ 87%]
tests/unit/test_skills.py::test_peak_richness_question_is_routed PASSED [ 90%]
tests/unit/test_skills.py::test_abundance_question_is_routed PASSED [ 93%]
tests/unit/test_skills.py::test_effort_question_is_routed PASSED [ 96%]
tests/unit/test_skills.py::test_unsupported_question_is_not_routed PASSED [100%]

===== 33 passed in 3.47s =====

```